message to the console while also persisting the logs to a file.

TIP By default, the rolling file provider writes logs to a subdirectory of your application. You can specify additional options such as filenames and file size limits using overloads of `AddFile()`. For production, I recommend using a more established logging provider, such as Serilog.

The key takeaway from listing 26.5 is that the provider system makes it easy to integrate existing logging frameworks and providers with the ASP.NET Core logging abstractions. Whichever logging provider you choose to use in your application, the principles are the same: add a new logging provider to `WebApplicationBuilder.Logging` using extension methods like `AddConsole()`, or `AddFile()` in this case.

Logging your application messages to a file can be useful in some scenarios, and it's certainly better than logging to a nonexistent console window in production, but it may still not be the best option.

If you discovered a bug in production and needed to look at the logs quickly to see what happened, for example, you'd need to log on to the remote server, find the log files on disk, and trawl through them to find the problem. If you have multiple web servers, you'd have a mammoth job to fetch all the logs before you could even start to tackle the bug—assuming that you even have remote access to the production servers! Not fun. Add to that the possibility of file permission or drive space problems, and file logging seems less attractive.

Instead, it's often better to send your logs to a centralized location, separate from your application. Exactly where this location may be is up to you; the key is that each instance of your app sends its logs to the same location, separate from the app itself.